

Introduction to Security

Musard Balliu

Lecture 1

DD2395/DD2391

Today's Plan

- **Course overview and organization**
- **Introduction to computer security**
- **Security mindset and principles**

Course Team

Instructors

Musard Balliu



Sonja Buchegger



Roberto Guanciale

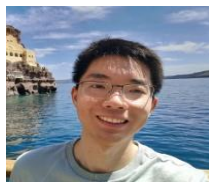


Teaching Assistants (TAs)

Eric Cornelissen



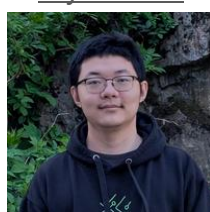
SiKai Lu



Frank Reyes García



Ziyuan Jia



Mojtaba Moazen



Antonio Napoli



Tom Sorger



About Me

- DD2391/DD2395: Course responsible, teacher, examiner
- 2022 - now: Associate Professor at EECS/TCS
- 2018 - 2021: Assistant Professor at KTH
- 2014 - 2017: PostDoc at Chalmers
- 2009 - 2014: Ph.D. at KTH
- Web page: <https://people.kth.se/~musard>
- I do research in language-based security, web and mobile app security, IoT app security, computer security foundations, and software supply chain security



Course Overview

What is cyber/computer security?

Computer Security

- Computer security studies how systems behave in the presence of an **adversary** (an intelligence that actively tries to cause the system to misbehave).
- Goal: Prevent bad things from happening
 - Confidential information leaked
 - Important information damaged
 - Critical service unavailable
- Computer security is important, hard, and fun

Why is security important?

It is important for our

- physical safety
- confidentiality/privacy
- functionality
- protecting our assets
- successful business
- a country's economy and safety
- (space for own notes)

Let's check it out:

- <https://www.theregister.com/security/>
- <https://www.wired.com/category/security/cyberattacks-hacks/>

Why is security important?

Privacy/Confidentiality

Twitter 2018:

330,000,000

Capital One 2019:

100,000,000

Facebook 2021:

533,000,000

user records breached

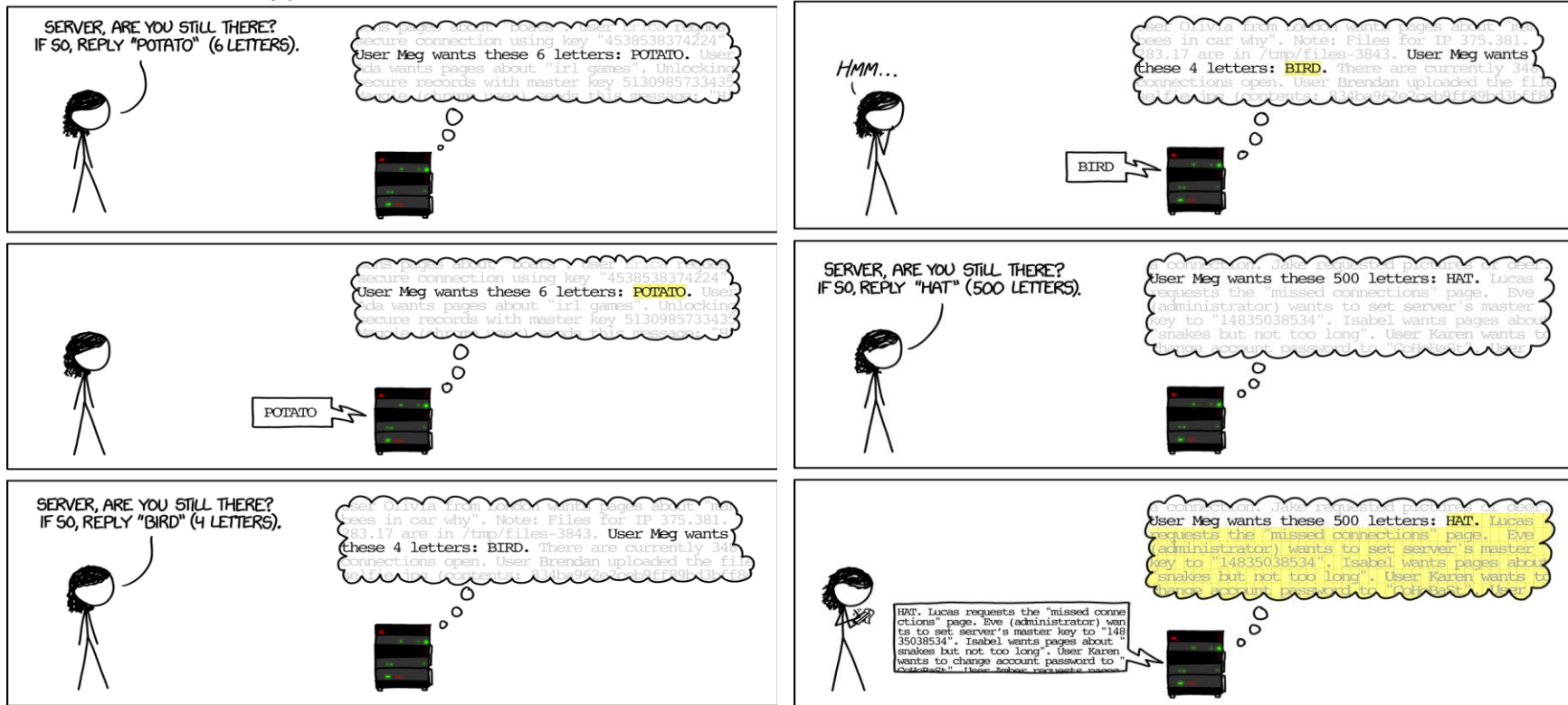
In 2020, there were over 1001 breaches, affecting the data of 155,000,000 individuals

Why is security important? - Heartbleed



Why is security important? - Heartbleed

HOW THE HEARTBLEED BUG WORKS:



Why is security hard?

Web app

I wrote the app, therefore it's secure...



Why is security hard?

I wrote the app, therefore it's secure...

Web app

Browser

OS

CPU



We wrote the browser, and therefore it's secure

We wrote the OS, and therefore it's secure

We designed the CPU, and therefore it's secure

How likely the whole stack is secure?

Web app

Browser

OS

CPU

Informal math

Probability that the Web app has a security bug = p_1

Probability that the Web app is secure = $1 - p_1$

...

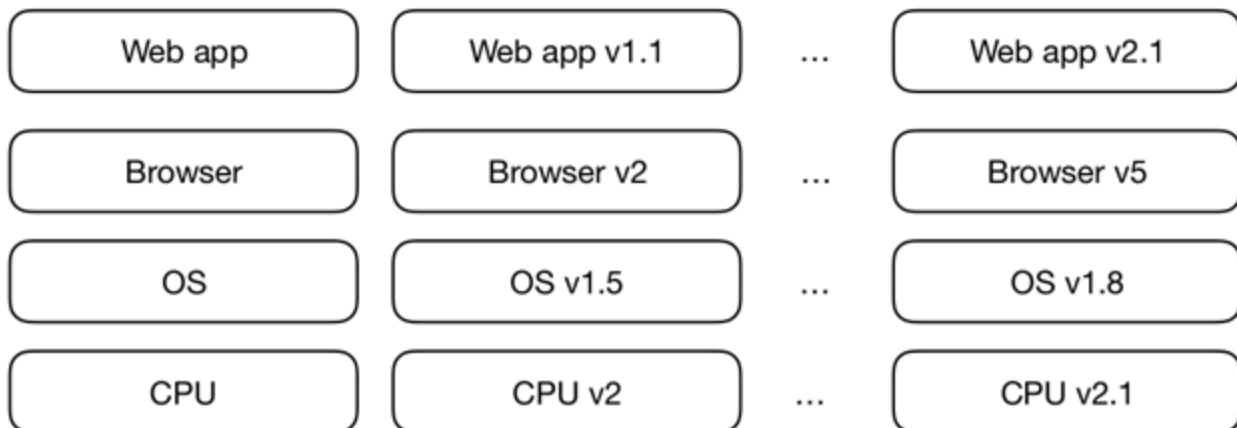
Probability that the CPU has a security bug = p_4

Probability that the CPU is secure = $1 - p_4$

Probability that the "whole stack" is secure

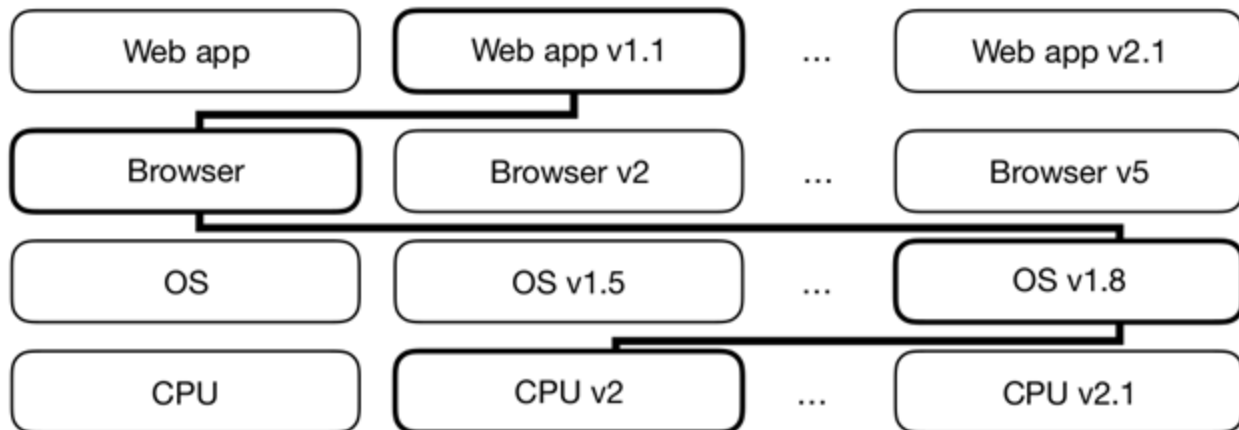
$$(1 - p_1) \cdot (1 - p_2) \cdot (1 - p_3) \cdot (1 - p_4)$$

Why is security hard?



Analyzing, or even anticipating, all possible combinations is much harder

Why is security hard?



Analyzing, or even anticipating, all possible combinations is much harder
Eventually, there is a crack somewhere...

General Goals

- How to think about computer systems in presence of adversaries/attackers
 - How to identify and exploit threats, vulnerabilities and their significance
 - How to build computer systems with robust security properties
 - How to evaluate defenses and limitations provided by today's systems
 - How attacks and defenses work in practice
 - How to present security concepts to others
-
- What mistakes *not to make!*

Course Overview

- Introduction to Security
 - What are some general models and principles when thinking about security?
- Cryptography
 - How to securely send information over an insecure channel?
- Authentication and Authorization
 - How to bind principals to actions and govern whether actions are permitted?
- Web Security
 - What is the web security model and how to defend against attacks on the web?
- Network Security
 - What are some attacks on the Internet, and how to defend against them?
- Memory Safety
 - How do attackers exploit insecure software? How to defend against these attacks?
- Privacy and Human Aspects
 - How protect privacy on the Internet? How do human aspects affect security?

Course Organization

Course Structure - Overview

- Lectures: twice a week
 - In-person, no recording provided
 - Slides and preparation/reading material provided in Canvas
 - Some lectures will require preparation, e.g., watching short videos
 - Attendance is not mandatory, but important for exam and assignments
- Lab assignments: 3 mandatory assignments and 1 optional
 - Lab assignment 1: Secure email with GPG (Lab G) - individual and automatically-graded
 - Lab assignment 2: Web attacks and defenses (Lab W) - groups of 3 students with presentation, 1 merit point
 - Lab assignment 3: Buffer overflow vulnerabilities (Lab O) - individual and automatically-graded, 1 merit point
 - Lab assignment 4 (optional): Zip bomb (Lab Z) - individual and automatically-graded, 1 merit point
- Exercises: practice concepts covered in lectures
 - 4 mandatory exercises – individual, automatically-graded, collaboration is allowed, also in bigger groups
 - Get together in lab sessions to collaborate and do exercises
- Seminar DD2395 - presentation of security topics – group of 3 students (same as Lab W), mandatory
- Project - only for students in DD2391 and DD2394, 4 students, mandatory
 - The project includes the seminar
 - Roberto Guanciale will provide details on Monday, August 31st, 9:00-10:00

Lab Sessions

- Goal: support students' work on lab assignments and exercises
- Yes, there are many lab sessions scheduled, but you can choose which session to attend on need basis
- Attendance is not mandatory, except for presentation of lab assignment 2 and seminar/project
- Activities
 - Getting together to work on exercises: collaboration is allowed and encouraged
 - Seek help from TAs for lab assignments
 - Present lab assignment 2 (Web)
 - Present seminar topic (DD2395) or project (DD2391 and DD2394)
- In-person attendance, no Zoom or similar
 - Exceptions require strong motivation and should be agreed with course responsible (Musard)
- Use KTH queue to seek help from TAs <https://queue.csc.kth.se/Queue/Dasak>
 - First-come first-serve policy

Communication and Feedback

- Course web page in Canvas: <https://canvas.kth.se/courses/64038>
- Discussion forum: https://canvas.kth.se/courses/64038/discussion_topics
 - Organized by topic. If a topic is not there, create your own.
- Announcements: <https://canvas.kth.se/courses/64038/announcements>
 - A summary of "Activities of the week" will be shared
- Course email address: security-dd239x@kth.se
 - Only if your question is sensitive, e.g, reveals solutions, otherwise use Discussion forum above
 - TAs and instructors have access to it. If questions too sensitive, email musard@kth.se
- Ask in person during lectures, breaks, lab sessions, office hours
 - **No communication via Canvas messages is supported**
- Soft protocol for course activities
 - Questions about course/lectures → Ask course responsible and instructors
 - Questions about assignments (labs, exercises, seminar) → Ask TAs
- Course feedback
 - Direct feedback to course responsible, instructors, and TAs
 - Anonymous feedback via course representatives. Any volunteers?
 - Final course evaluation

Frequently Asked Questions - FAQs

We have compiled a list of FAQs about the course, which you should (have) check(d) out

<https://canvas.kth.se/courses/64038/pages/frequently-asked-questions-faq>

Prerequisites

- Knowledge and skills in programming
- Knowledge in foundations of computer science
- Internet protocols and principles - recommended
- Basics of command-line interface, Python, and operating systems – recommended
- Web programming: HTML, JavaScript, SQL – recommended
- C/C++ programming, debugging e.g., gdb – recommended
- Version control systems, e.g., Git – recommended

Resources and Groups

- The course does not have an official textbook, but the following books can be helpful
 - [Security Engineering, Ross Anderson, 3rd edition](#)
 - [Computer Security, Dieter Gollmann](#)
 - Freely available online within KTH network
- Course website: <https://canvas.kth.se/courses/64038>
 - Course schedule, lecture slides, reading/preparation material, assignments, and other resources are all posted there
- Some assignments are carried out in groups of students
 - Find your group partners and sign up at <https://canvas.kth.se/courses/64038/users>
 - DD2395: Lab assignment 2 and Seminar, 3 students : [DD2395-Lab W-Seminar](#)
 - DD2391: Lab assignment 2, 3 students : [DD2391-Lab W](#)
 - Project, 4 students: More details from Roberto on August 31

Examination and Grading

- The course has a final written exam
 - Exam: **October 19th, 14:00-18:00.**
 - Re-exam: **December 16th, 14:00-18:00**
- Past exams will be available at <https://canvas.kth.se/courses/64038/pages/past-exams>
 - Structure will be similar to 2023/2024-2025/2026
 - Grading criteria will be updated – can still skip one-two tasks and get a top grade
 - Students are allowed to bring at most 10 A4 papers (20 pages in total with student's full name written on every page) of handwritten or printed material to the exam.

DD2395 Computer Security

- LAB (lab assignments, exercises, and seminar): Pass/Fail
- Written exam: A-F
- Final course grade:
 - Same as written exam, if the written exam grade is C or less.
 - B, if the student has earned 1 merit point and written exam grade is A or B
 - A, if the student has earned 2 merit points and written exam grade is A
 - The above assumes that the student has passed LAB.

DD2394 Cybersecurity project

- Project: Pass/Fail
- Final course grade: Pass/Fail (same as the grade of project)

DD2391 Cybersecurity Overview

- LAB (lab assignments, exercises, and project): Pass/Fail
- Project: Pass/Fail
- Written exam: A-F
- Final course grade:
 - Same as written exam, if the written exam grade is C or less.
 - B, if the student has earned 1 merit point and written exam grade is A or B
 - A, if the student has earned 2 merit points and written exam grade is A
 - The above assumes that the student has passed LAB and Project.

Deadlines, Deadlines, Deadlines

- The course has both soft and hard deadlines for different activities
 - Soft deadlines: Helps you keep up to pace with the course – highly recommended, TA support guaranteed.
 - Hard deadlines: If you miss it, you fail the course activity. Check [FAQs](#) for details
- If you miss a soft deadline, make sure you complete the activity by the hard deadline.
- If you miss a hard deadline, you'll get a chance to complete the missing assignment in June 2027 (lab week) with no merit points. If you miss or fail the exam, the re-exam is scheduled on December 16th, 14:00 – 18:00.
- Students that receive grade Fx can complement with an oral discussion to earn grade E. [Grade raising \(known as plussning\)](#) is **NOT allowed** for these courses.
- Deadlines will be announced in Canvas under [Assignments](#). Check out [FAQs](#) for an overview of hard deadlines.

Code of Honor and Ethics

- Course AI policy: <https://canvas.kth.se/courses/64038/pages/policy-on-use-of-ai-powered-tools-in-graded-course-activities>
- EECS code of honor: <https://www.kth.se/en/eecs/utbildning/hederskodex>
- As a computer security course, we view potential cheaters as “attackers.”
 - Cheating includes exchanging solutions between students/groups, not stating true contributions within a group, not citing the sources, and violating course AI use policy.
 - We may use tools to detect cases of academic dishonesty and at minimum you will fail the assignment.
 - If you take the course honestly, you don't need to worry about these!
- Ethics: With great power come great responsibility
 - You will learn a lot about attacks out of necessity. To be able to defend against the attacker, you must learn the techniques that attackers use. It is usually OK to break into **your own** systems - this is a great way to evaluate your own systems.
 - It is *unethical* and *criminal* to break into someone else's systems **without their permission**

What is cyber/computer security?

Defining security

A computer system is *secure* when it

- does what it should
- and nothing more.

A security *policy* stipulates what should and should not be done.

A *principal* is an entity who can take actions

- person
- program
- system
- ...

Not to be confused with *principle*—a fundamental truth or basis (more later)

Security Policies

- "The system shall prevent/detect *action* on/to/with *asset*."
 - e.g., "The system shall prevent theft of money"
 - e.g., "The system shall prevent erasure of account balances"
- Specify **what** not **how**
- Poor goals:
 - "the system shall use encryption to prevent reading of messages"
 - "the system shall use authentication to verify user identities"
 - "the system shall resist attacks"

Security policies typically formulated in terms of three security properties

Security Properties

- **Confidentiality:** protection of assets from unauthorized disclosure
 - **Assets:** information, resources, ...
 - **Disclosure:** to a person, a program, a system, ...
 - *Privacy* concerns information about individuals (people, organizations, etc.)
 - Confidentiality policy: Keep contents of a file from being read (access control)
- **Integrity:** protection of assets from unauthorized modification
 - what changes are allowed to system and its environment, including inputs and outputs
 - Integrity policy: Only certain principals may write to a file (access control).
 - Output is correct according to some specification
- **Availability:** protection of assets from loss of use
 - what should to happen when/where. Denial of Service affects availability
 - Availability policy: Operating system must accept inputs periodically.
 - Program must produce output by specified time

Examples

Example 1

- **Attack:** John copies Mary's homework
- What is a **security property** this attack would violate?

Example 2

- **Attack:** Paul causes Linda's system to freeze
- What is a **security property** this attack would violate?

The Security Mindset

Attacker's Mindset

- Understand techniques for circumventing security
- Look for ways security can break, not reasons why it won't



Defender's Mindset

- Know what you are defending and against whom
- Risk analysis
- Benefits vs. costs: No system is ever completely secure
- Rational paranoia

Attacker's Mindset

- Look for weakest links
- Identify assumptions that security depends on. Do assumptions hold?
- Think outside the box. Don't be constrained by system designer's worldview.
- Practice: When you interact with a system, think about what it means to be secure, and how it might be exploited.

Exercises (Don't try any of these:):

- Breaking into KTH buildings
- Passing course without studying
- Cheating at a written exam
- Stealing my password



Defender's Mindset

- Security policy
 - What assets are we trying to protect
 - What security properties are we trying to enforce
- Threat model
 - Who are the attackers? Capabilities? Motivations?
 - What kind of attack are we trying to prevent?
- Risk Model
 - What are the weakest links of the system
 - How likely is an attack and what will it cost
- Countermeasures
 - What mechanisms can we use to protect the assets
 - Costs vs benefit trade-offs
 - Technical vs nontechnical
- Common mistake: Try to convince yourself that the system is secure
- Better approach: Identify weaknesses and focus on correcting them

Security Model

- System model – how does the system behave?
- Security policies – what do we want from the system?
- Threat model – what exactly can attacker do?
- Trust – what components do we trust?
- Security mechanisms (countermeasures) – how do we implement security?
- Assurance - does it really work

System Model

- If we can't even describe how the system works, it's pointless to reason about its security
- Examples of (formal) models
 - Trace models
 - Sequence of actions in a network protocol
 - A sequence of system calls in an OS execution
 - Models based on programming language (PL) semantics – these allow us to formally characterize behavior of programs

Security Policies

- Security Policy
 - What is (what is not) allowed
 - Who is allowed to do what
- CIA triad: Confidentiality, Integrity, Availability
- Other
 - Privacy, Accountability, ...

Threat Model (Attacker Model)

- Who is the attacker and what exactly can an attacker do
- What can an attacker observe
 - Final output, I/O events, parallel memory access, timing, power consumption
- How can an attacker influence the system?
 - Attacker can provide part of the input
 - often times, the attacker is just another user/principal
 - Co-resident attackers that not only observe but also influence the runtime
 - CPU caches, Garbage Collector, browser event queue, network congestion
- Attacker may provide the code that we run in our system. Examples?
- Attacker can and will get lucky

The Parable of the Bear Race



“I don’t have to outrun the bear. **I just have to outrun you.**”

Takeaway: You often just need to have “good enough” defense to make attackers turn somewhere else.

Trust and Trustworthiness

Trust: the degree to which a principal is expected to behave. **It's an assumption!**

- What the principal is expected to do? Example: Change password every 3 months
- What the principal is not expected to do? Example: Expose the password
- A trusted component **is assumed** to satisfy a security policy
- Trusted Computing Base (TCB): components whose failure compromises the security of a system, i.e., components of a system that security relies upon.
- Trustworthy component: enjoys evidence that it satisfies the security policy

TCB example

Goal: only authorized users are allowed to log into a system using SSH

What is the TCB:

- SSH daemon
- Operating system
- CPU

Examples of what is not in the TCB:

- A video streaming application installed on the same machine

Security Mechanisms (Countermeasures)

A defense that protects against attacks by neutralizing either the threat or vulnerability involved

Strategy:

- **Prevent:** block attack or close vulnerability
- **Deter:** make attack harder but not impossible
- **Deflect:** make other targets more attractive
- **Mitigate:** make harm less severe
- **Detect:** as it happens or after the fact
- **Recover:** undo harm

Mechanisms:

- Cryptography, authentication, access control, isolation, sandboxing, monitoring, information flow control, security testing, fuzzing, dynamic analysis, static analysis, verification

Security Assurance

Do our security mechanisms enforce our security policies?

- Trusted Computing Base (TCB): the set of all system components that must be trusted in order to maintain the overall system security
- Question to ask: Can we reduce the security of the system to the security of the trusted computing base?
 - Yet, if the attacker can violate our requirements, then our TCB must be broken
- Important to keep TCB small
 - KISS principle: Keep It Simple, Stupid
- Formal reasoning, Testing, ...

Summary: Security Model

- System model – how does a system behave?
- Security policies – what do we want from the system?
- Security mechanism – how do we implement security
- Threat model – what are the adversary's capabilities?
- Trust – whom do we trust?
- Assurance – does it really work?

Security Principles

Security Principle: Accountability

Hold principals responsible for their actions

- **Authentication:** mechanisms that bind principals to actions
 - **Authorization:** mechanisms that govern whether actions are permitted
 - **Audit:** mechanisms that record and review actions
-
- Also known as the Gold Standard
 - More in lecture 3-4



Security Principle: Complete Mediation

Every operation requested by a principal must be intercepted and determined to be acceptable according to the security policy

- Ensure that every access point is monitored and protected
- **Reference monitor:** Single point through which all access must occur
 - Example: A network firewall, airport security, the doors to the dorms
- Component that does the interception and determination is the reference monitor
- Reference monitor should be part of TCB
- Related to Accountability

Complete Mediation: Spot the Issue



Security Principle: Least Privilege

Principals should be given the minimum privileges necessary to accomplish their task

- Consider what permissions an entity or program *needs* to be able to do its job correctly
 - If you grant unnecessary permissions, a malicious or hacked program could use those permissions against you
- Limits the damage that can result from accident or malice
- Related to "need to know"
- Drawbacks?

Least Privilege: Flashlight app

Android Flashlight App Developer Settles FTC Charges It Deceived Consumers

'Brightest Flashlight' App Shared Users' Location, Device ID Without Consumers' Knowledge

"When consumers are given a real, informed choice, they can decide for themselves whether the benefit of a service is worth the information they must share to use it," said Jessica Rich, Director of the FTC's Bureau of Consumer Protection. "But this flashlight app left them in the dark about how their information was going to be used."



Security Principle: Failsafe Defaults

Base decisions on the presence of privilege, not the absence of prohibition

- The default answer is "no"
- Say "yes" only when there is an explicit reason to do so
- Principals who discover they don't have access will complain
- Attackers who discover they do have access won't complain!
- Allowlists better than denylists, though may hinder usability

Security Principle: Separation of Privilege

Different operations should require different privileges

- Supports Least Privilege Principle, but hinders usability
- Disseminate privileges for an operation amongst multiple principals (Separation of Duty)
- If you need to have a privilege, consider requiring multiple parties to work together (collude) to exercise it
 - It's much more likely for a single party to be malicious than for all multiple parties to be malicious and collude with one another

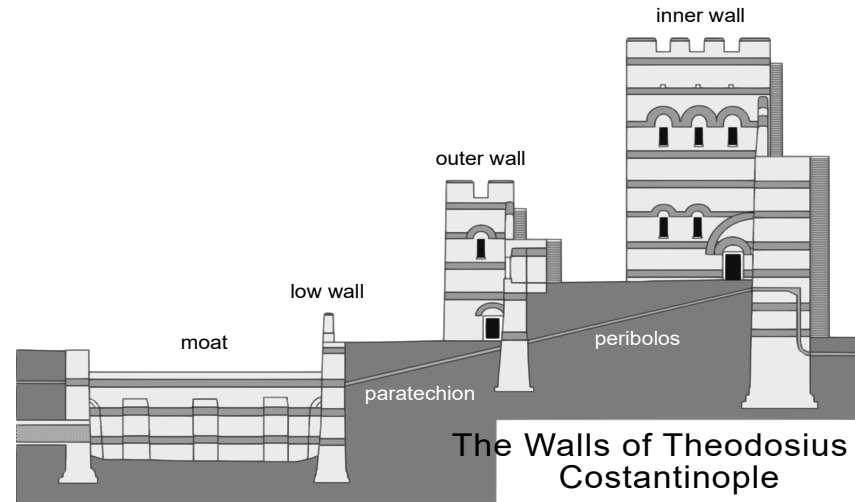
Security Principle: Defense in Depth

Prefer a set of complementary mechanisms over a single mechanism

- **Independent:** attack that compromises one mechanism is unlikely to compromise others
- **Overlapping:** attacks must compromise multiple mechanisms to succeed
- Cf. Authentication, authorization, audit
- However, defenses are costly

The Theodosian Walls of Constantinople

- The ancient capital of the Byzantine empire had a wall...
 - Well, they had a moat...
 - then a wall...
 - then a depression...
 - ... and then an even bigger wall
- It also had towers to rain fire and arrows upon the enemy...
- **Takeaway:** Defense in depth



Security Principle: Economy of Mechanism

Prefer mechanisms that are simpler and smaller

- Easier to understand, construct, analyze
- Hence less likely to have unknown vulnerabilities
- Applies to any aspect of system, not just security
- Keeps the TCB small

Security Principle: Open Design

Security shouldn't depend upon the secrecy of design or implementation

- Don't Rely on Security Through Obscurity
- Always assume that the attacker knows every detail about the system you are working with (algorithms, hardware, defenses, more).

Security Principle: Open Design

Security shouldn't depend upon the secrecy of design or implementation

Arguments **for** open design

- Secrets eventually come out: reverse engineering is possible, employees move around
- Making details public increases chance of identifying and repairing vulnerabilities

Security Principle: Open Design

Security shouldn't depend upon the secrecy of design or implementation

Arguments **against** open design

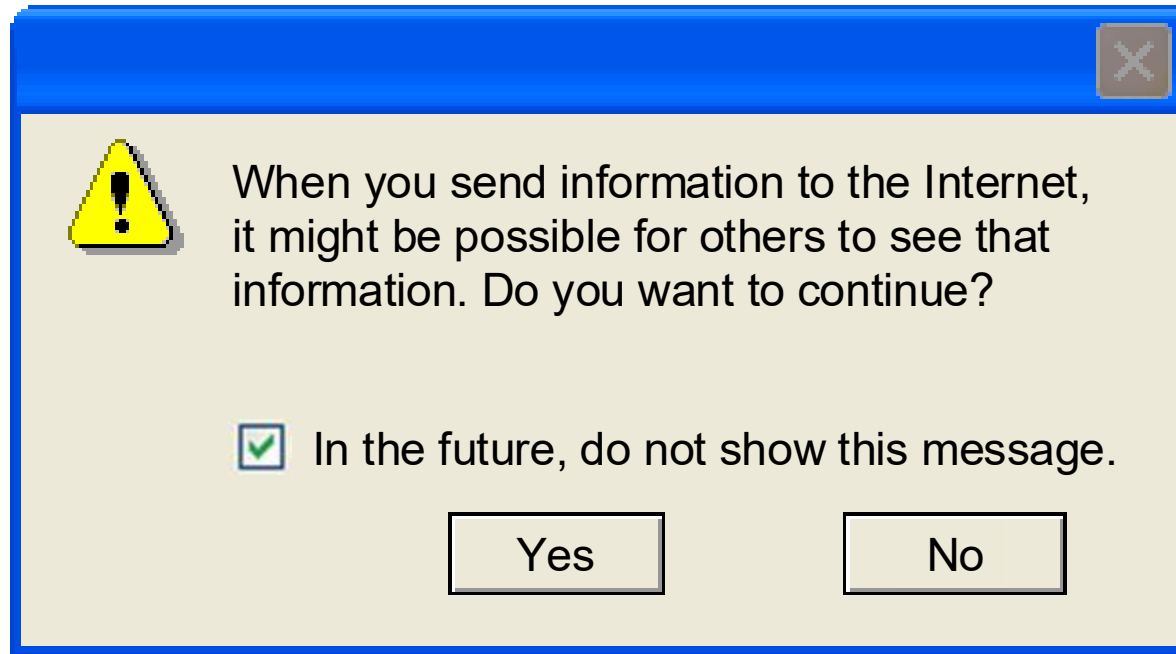
- Secrecy supports Defense in Depth by making it harder to find vulnerabilities
- Lack of hard evidence that Linus' Law really holds ("given enough all eyeballs, all bugs are shallow")
- After identification, some vulnerabilities cannot quickly or easily be repaired, cf. Spectre

Security Principle: Psychological Acceptability

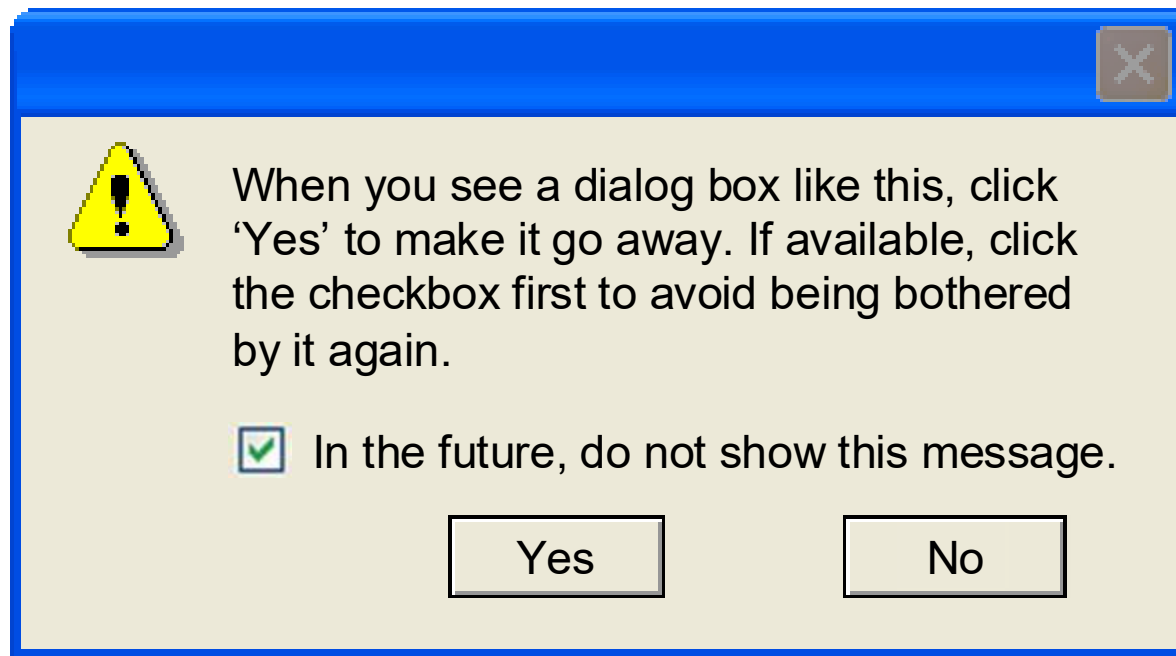
Minimize the burden of security mechanisms on humans

- Don't make operations (much) more difficult to complete than if security mechanisms were absent
- Don't make configuration difficult
- Produce comprehensible error messages
- Seek right tradeoff between security and usability

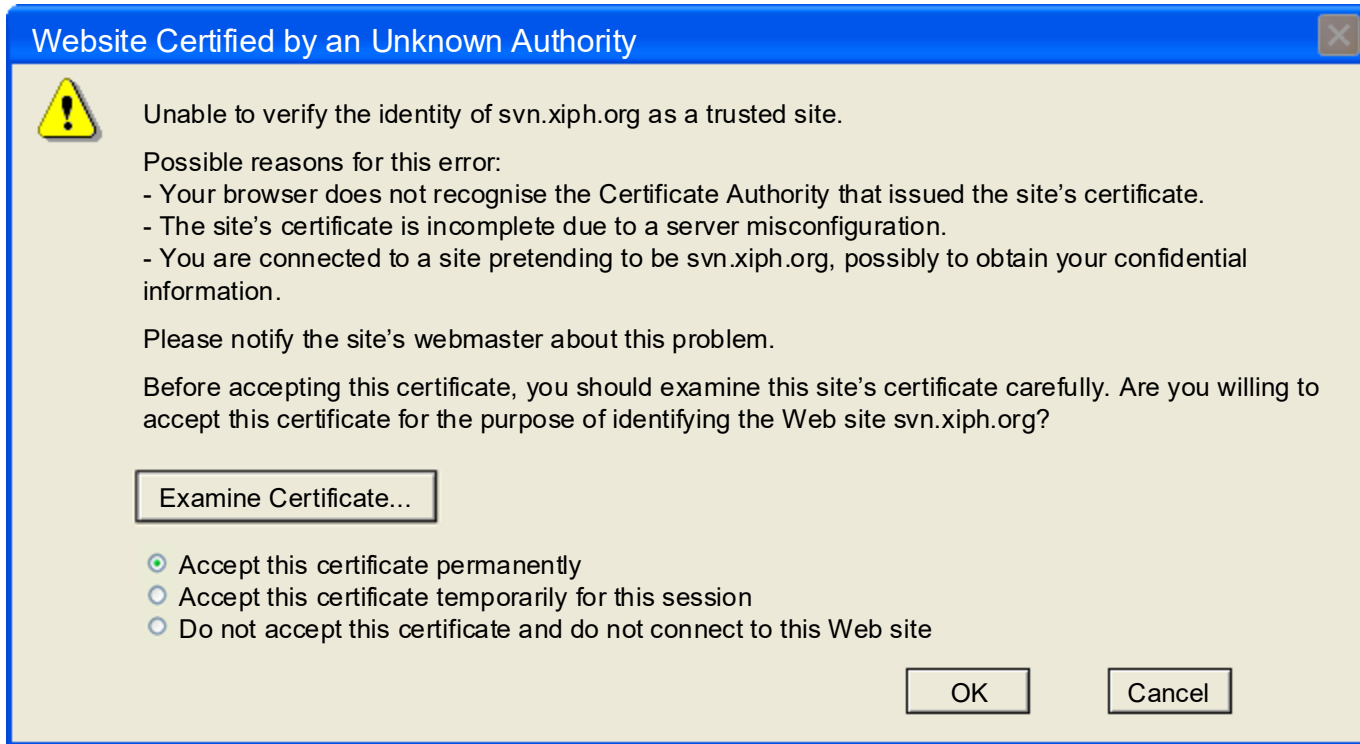
Warning Dialogs



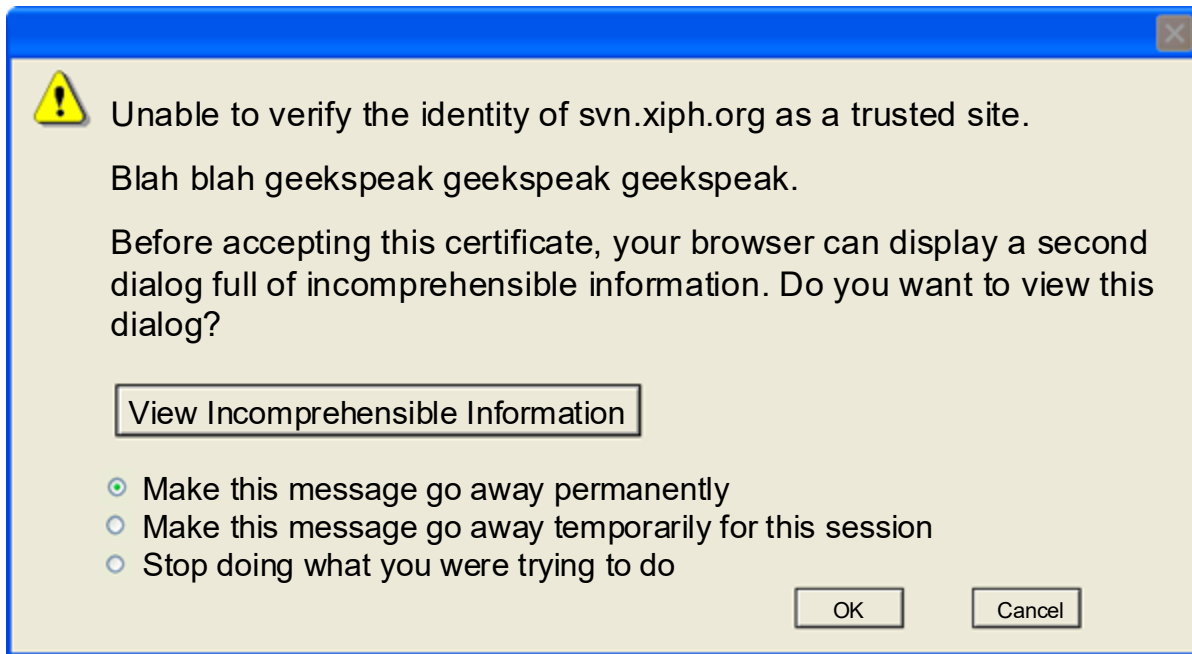
Warning Dialogs



Warning Dialogs



Warning Dialogs



The presence of warning dialogs often represent a failure: How is the user supposed to know what to do?

Takeaway: Consider human factors

Security Principle: Psychological Acceptability

- It all comes down to people: The users
 - Users like convenience (ease of use)
 - If a security system is unusable, it will be unused
 - Users will find way to subvert security systems if it makes their lives easier
- It all comes down to people: The programmers
 - Programmers make mistakes
 - Programmers use languages and tools that allow them to make mistakes (e.g. C and C++)
 - AI exacerbates the problem further – programmer can be an LLM
- It all comes down to people: Everyone else
 - Social engineering attacks exploit other people's trust and access for personal gain
- Consider the tools presented to users, and make them **fool-proof**



Physical security keys use the fact that humans are trained to safeguard keys

Homework

- Check out the reading material for Lecture 1 and read the suggested articles
- Log in to <https://gits-15.sys.kth.se/>